

INFORME_PROYECTO



Leonardo Araque

Deep learning
Universidad de Antioquia
Julian David Arias - Raul Ramos Pollan
2026 - 1

1. Introducción

El objetivo de este proyecto fue desarrollar un sistema de clasificación multiclase capaz de predecir la raza de una mascota a partir de una imagen, utilizando técnicas de visión por computador y deep learning. Para ello se eligió el Oxford-IIIT Pet Dataset, un conjunto de datos ampliamente utilizado en tareas de clasificación fina de imágenes, que contiene 37 categorías de razas de perros y gatos con alta variabilidad en pose, escala, fondo e iluminación.

La propuesta del proyecto se estructuró como una secuencia de experimentos progresivos. Primero se realizó una exploración del dataset y se construyó un pipeline de preprocesado reproducible. Posteriormente se implementó una línea base mediante una CNN sencilla entrenada desde cero. Después se desarrolló una solución basada en transfer learning con MobileNetV2 preentrenada en ImageNet. Finalmente, se aplicó fine-tuning parcial sobre la red preentrenada para adaptar mejor el extractor de características al dominio específico del dataset.

La motivación principal fue comparar tres niveles de complejidad: una arquitectura propia simple, una red preentrenada congelada y una red preentrenada ajustada finamente. Esta secuencia permite justificar la evolución de la solución y analizar de manera clara el impacto del preprocesado, la transferencia de aprendizaje y el fine-tuning en el rendimiento final del sistema.

2. Estructura de los notebooks entregados

El repositorio final fue organizado con notebooks numerados, tal como se solicitó en la guía de la entrega. Cada notebook cumple una función específica dentro del flujo completo del proyecto.

Notebook	Propósito	Salida esperada
01 - exploracion de datos.ipynb	Inspección del dataset, revisión de clases, visualización de imágenes y análisis exploratorio.	Comprensión del problema, distribución de clases y observaciones para el pipeline.
02 - preprocesado.ipynb	Construcción del pipeline de datos para CNN base y MobileNetV2.	Datasets listos para entrenamiento, validación y prueba.
03 - arquitectura de linea de base.ipynb	Implementación de una CNN simple entrenada desde cero.	Línea base cuantitativa y cualitativa.
04 - transfer learning.ipynb	Uso de MobileNetV2 congelada como extractor de características.	Primera solución fuerte del proyecto.
05 - fine tuning y evaluacion final.ipynb	Descongelamiento parcial de MobileNetV2 y evaluación final.	Modelo final, métricas finales y análisis de errores.

La separación en notebooks permite una presentación ordenada, reproducible y coherente con la metodología del proyecto. Además, facilita mostrar claramente las iteraciones realizadas y la evolución de la solución.

3. Datos y forma de acceso

Se utilizó el Oxford-IIIT Pet Dataset, disponible de forma pública y accesible desde TensorFlow Datasets. De acuerdo con la documentación oficial del dataset, contiene 37 categorías de razas, aproximadamente 200 imágenes por clase y un total de 7349 imágenes. En TensorFlow Datasets aparece dividido en 3680 ejemplos de entrenamiento y 3669 ejemplos de prueba. Además de la etiqueta de raza, el dataset incluye la especie, máscaras de segmentación y cajas de cabeza, aunque en este proyecto se empleó únicamente la tarea de clasificación de raza. [Ref. 1, 2]

Una ventaja importante para la reproducibilidad es que los datos se cargan directamente desde TensorFlow Datasets mediante el identificador `oxford_iiit_pet`. Esto evita depender de rutas locales y permite que los notebooks se ejecuten en Google Colab desde cero. En los notebooks 02, 03, 04 y 05 se utilizó la siguiente estrategia de partición:

- `train[:80%]` para entrenamiento
- `train[80%:]` para validación
- `test` para evaluación final

Esta decisión permite usar el split original de entrenamiento para derivar un conjunto de validación sin tocar el conjunto de prueba, que se reserva exclusivamente para la evaluación final.

Código base utilizado para obtener el dataset:

```
(train_raw, val_raw, test_raw), info = tfds.load(
    'oxford_iiit_pet',
    split=['train[:80%]', 'train[80%:]', 'test'],
    with_info=True,
    as_supervised=False
)
```

4. Descripción de la solución

La solución completa se diseñó en tres niveles: una línea base con CNN propia, una solución con transfer learning y una versión final con fine-tuning. En todos los casos se partió de un pipeline de preprocesado común y de métricas de clasificación multiclase.

4.1 Preprocesado

Las imágenes del dataset presentan tamaños variables, por lo que fue necesario redimensionarlas a una forma común de 224x224 píxeles. Para la línea base se aplicó una normalización simple al rango [0,1]. Para MobileNetV2 se empleó el preprocesado oficial de la arquitectura mediante `tf.keras.applications.mobilenet_v2.preprocess_input`, que adapta la distribución de los valores de entrada al rango esperado por el modelo preentrenado.

También se incorporó data augmentation moderado mediante `RandomFlip` horizontal, `RandomRotation(0.1)` y `RandomZoom(0.1)`. Estas transformaciones se eligieron para mejorar la generalización del modelo sin distorsionar en exceso la morfología de la mascota.

4.2 Línea base: CNN entrenada desde cero

La arquitectura base se construyó con varios bloques Conv2D + MaxPooling, seguidos de una capa GlobalAveragePooling2D, una capa densa intermedia con activación ReLU, dropout y una capa final softmax de 37 salidas. Esta red funciona como punto de comparación metodológico: no está pensada para ser la mejor solución, sino para ofrecer una referencia clara de desempeño cuando se entrena una CNN simple directamente sobre el dataset.

4.3 Transfer learning con MobileNetV2

La segunda iteración utilizó MobileNetV2 preentrenada en ImageNet como extractor de características. Se eliminó la cabeza original de clasificación con `include_top=False`, se congeló la base convolucional (`base_model.trainable = False`) y se añadió una nueva cabeza compuesta por GlobalAveragePooling2D, dropout y una capa Dense con 37 salidas softmax. En esta fase solo se entrenó la cabeza nueva, mientras la base preentrenada aportaba representaciones visuales ya aprendidas. [Ref. 3]

4.4 Fine-tuning

La versión final partió del modelo con transfer learning y desbloqueó parcialmente la base de MobileNetV2 para ajustar mejor sus características al dominio del Oxford-IIIT Pet Dataset. Para ello se activó `base_model.trainable = True`, se mantuvieron congeladas las primeras capas y se entrenaron capas más profundas con una tasa de aprendizaje reducida. También se mantuvieron congeladas las capas BatchNormalization por estabilidad. Esta estrategia busca refinar las representaciones visuales sin destruir el conocimiento previo adquirido durante el preentrenamiento. [Ref. 4]

5. Iteraciones realizadas

El proyecto no se resolvió con un único experimento, sino mediante una serie de iteraciones que permitieron comparar enfoques y justificar la solución final.

Iteración	Modelo	Objetivo	Hallazgo esperado
1	CNN base	Establecer una referencia inicial.	Desempeño aceptable, pero inferior a soluciones preentrenadas.
2	MobileNetV2 congelada	Aprovechar conocimiento previo de ImageNet.	Mejora clara en accuracy y estabilidad.
3	MobileNetV2 con fine-tuning	Adaptar la base al dominio del dataset.	Mejora adicional o refinamiento del resultado final.

Este diseño experimental permite que el informe final no solo muestre un resultado, sino una evolución técnica y argumentada de la solución.

6. Resultados

Como este informe fue preparado antes de ejecutar los notebooks en el entorno final de entrega, los campos numéricos deben actualizarse con los resultados reales obtenidos en Colab. Aun así, la estructura de reporte queda lista para documentar los hallazgos de manera ejecutiva y coherente.

6.1 Resultados cuantitativos

La comparación principal debe realizarse entre la línea base, el modelo de transfer learning y el modelo final con fine-tuning. La tabla siguiente resume los campos mínimos que deben reportarse.

Modelo	Val. Accuracy	Test Accuracy	Val. Loss	Test Loss	Observación
CNN base	0.0611	0.0548	3.4854	3.5553	Sirve como referencia inicial.
MobileNetV2 congelada	0.8927	0.8942	0.3324	0.3425	Debe superar a la línea base.
MobileNetV2 + fine-tuning	0.9008	[COMPLETA R]	0.3150	[COMPLETA R]	Versión final del proyecto.

Adicionalmente, en los notebooks 03, 04 y 05 se generan métricas más detalladas como classification report, matriz de confusión y visualización de predicciones correctas e incorrectas. Estas herramientas son útiles para complementar la accuracy global y explicar qué clases tienden a confundirse con más frecuencia.

6.2 Resultados cualitativos

Además de los resultados numéricos, el análisis cualitativo debe apoyarse en las curvas de entrenamiento, la matriz de confusión y ejemplos visuales de predicción. En particular, conviene revisar:

- si el modelo base muestra señales de sobreajuste o subajuste;
- si MobileNetV2 converge más rápido y con mayor estabilidad;
- qué razas se confunden más entre sí;
- si el fine-tuning reduce errores entre clases visualmente parecidas.

Un comentario recomendable para el informe final es relacionar los errores con características visuales del dataset, por ejemplo: similitud en el color del pelaje, fondo dominante, pose del animal, ángulo de la foto o escala de la mascota dentro de la imagen.

7. Conclusiones

El proyecto demostró una secuencia de trabajo consistente para resolver una tarea de clasificación fina de imágenes. En primer lugar, la exploración de datos permitió identificar que el dataset es relativamente balanceado, pero visualmente desafiante por la similitud entre varias razas y por la variabilidad de fondo, pose e iluminación. En segundo lugar, el preprocesado estandarizado con

redimensionamiento, normalización y data augmentation permitió construir un flujo reproducible y apto para Google Colab.

La comparación entre una CNN entrenada desde cero y una red preentrenada puso en evidencia el valor del transfer learning para este tipo de problemas. Finalmente, el fine-tuning mostró una estrategia adecuada para refinar una arquitectura preentrenada y obtener la mejor versión del modelo.

Como trabajo futuro, el proyecto podría extenderse hacia tareas más avanzadas aprovechando las anotaciones adicionales del dataset, por ejemplo segmentación con U-Net o detección de objetos con bounding boxes. Sin embargo, para el alcance planteado en esta entrega, la formulación como clasificación multiclase con transfer learning y fine-tuning ofrece una solución sólida, justificable y alineada con los contenidos del módulo.

8. Reproducibilidad y consideraciones de entrega

Todos los notebooks se diseñaron para ejecutarse directamente en Google Colab. Para mantener la reproducibilidad se evitó depender de rutas locales y se cargó el dataset desde TensorFlow Datasets. El repositorio final debe incluir:

- los notebooks numerados;
- ENTREGA1.PDF;
- INFORME_PROYECTO.PDF;
- README.md con el enlace al video en YouTube.

9. Referencias

Parkhi, O. M., Vedaldi, A., Zisserman, A., & Jawahar, C. V. (2012). Cats and Dogs. IEEE Conference on Computer Vision and Pattern Recognition. Disponible en: <https://www.robots.ox.ac.uk/~vedaldi/assets/pubs/parkhi12cat.pdf>

TensorFlow Datasets. Oxford-IIIT Pet Dataset. Disponible en: https://www.tensorflow.org/datasets/catalog/oxford_iiit_pet

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Disponible en: <https://arxiv.org/abs/1801.04381>

TensorFlow Core. Transfer learning and fine-tuning. Disponible en: https://www.tensorflow.org/tutorials/images/transfer_learning